

Practical Work: Modular Arithmetic & Introduction to Cryptography

Done by: Muhammad Omais Rana

Exercise 1 — Understanding the Modulus

Consider the transformation: $C = (P + k) \bmod m$ with $P = 20$, $k = 7$, $m = 26$

Questions

1. Before using Python, calculate C by hand.

Answer: $C = (20+7) \bmod 26 \Rightarrow 27 \bmod 26 \Rightarrow 1$

2. What does $\bmod 26$ actually do to the result?

Answer: It wraps numbers around a system of 26 values (0 through 25) to map mathematical operations directly to the English alphabet.

3. What possible set of values can C take?

Answer: C can be any value in $\{0, 1, 2, \dots, 25\}$

4. Why does $m = 26$ make sense when working with the alphabet?

Answer: Same as I said previously, $m = 26$ matches the number of letters in the alphabet so each number 0–25 maps to exactly one letter.

5. Verify your answer using Python.

Answer: Added at end

Exercise 2 — Modular Addition Encryption

Consider the scheme: $C = (P + k) \bmod m$ with $m = 26$, $k = 7$

Encrypt the following plaintext values: $P \in \{0, 1, 5, 10, 19, 20, 25\}$.

Questions

1. Calculate all ciphertexts without using Python.

Answer: Taking “ $C = (P+7) \bmod 26$ ” into consideration,

- $P=0 \Rightarrow 7$ $*(0+7) \bmod 26 \Rightarrow 7$
- $P=1 \Rightarrow 8$
- $P=5 \Rightarrow 12$
- $P=10 \Rightarrow 17$
- $P=19 \Rightarrow 0$
- $P=20 \Rightarrow 1$
- $P=25 \Rightarrow 6$

2. Write a Python program to perform the encryption for all values.

3. Compare your manual calculations with the Python results.

Answer: They match

4. What happens mathematically when $P + k \geq 26$?

Answer: In the context of the given range, the operation subtracts the total added number from 26 and brings the excess back into range.

5. Why does the ciphertext always stay in the range 0 to 25?

Answer: Because “ $\bmod 26$ ” is designed to bring the remainder in that range.

6. Reflection: If someone knows $m = 26$ and the encryption formula $C = (P + k) \bmod 26$, what piece of information do they still need to decrypt your message?

Answer: They still need “ k ”. It’s the secret key

Exercise 3 — Discovering Decryption

Start with $C = (P + k) \bmod m$.

Questions

1. If encryption adds k , what mathematical operation should undo it? Derive the formula for P .

Answer: Subtraction of k will happen like this " $P = (C - k) \bmod m$ ".

2. Using $m = 26$, $k = 7$, and ciphertext $C = 1$:

(a) Calculate P by hand.

Answer: $P = (1 - 7) \bmod 26 \Rightarrow -6 \bmod 26 \Rightarrow 20$

(b) Verify your result using Python.

Answer: Added at end

(c) Re-encrypt your recovered P . Does it produce $C = 1$?

Answer: Yes. By re-encrypting like this, $C = (20 + 7) \bmod 26 \Rightarrow 1$

(d) Why does subtraction undo addition modulo m ?

Answer: Because subtraction is the inverse operation of addition as said in the first question.

3. Complete the following Python functions:

Answer: Added at end

Understanding Check

- Is the decryption key a completely different random number, or is it related to k ?

Answer: It's the same k , just used with subtraction instead of addition.

Exercise 4 — Modular Multiplication

Consider the encryption formula: $C = (P \times k) \bmod m$ with $P = 20$, $k = 5$, $m = 26$

Questions

1. Calculate C by hand.

Answer: $C = (20 \times 5) \bmod 26 \Rightarrow 22$

2. Verify using Python.

Answer: Added at end

3. Evaluate the encryption function for all $P \in \{1, 2, 3, \dots, 25\}$.

Answer: I'll make a simple table for this

P	1	2	3	4	5	6	7	8	9	10	11	12	13
C	5	10	15	20	25	4	9	14	19	24	3	8	13

P	14	15	16	17	18	19	20	21	22	23	24	25
C	18	23	2	7	12	17	22	1	6	11	16	21

4. Do different plaintext values ever produce the exact same ciphertext?

Answer: The ciphertext is distinct. It won't be very good if it produced the same text.

5. If two distinct plaintexts produce the same ciphertext, what problem does this create for decryption?

Answer: You won't be able to decrypt properly since you can't tell which original P produced that C.

Exercise 5 — How Do We Divide Modulo m?

Given $C = (P \times 5) \bmod 26$ and $C = 22$:

Questions

1. Why does $P = 22/5$ not work directly in modular arithmetic over integers?

Answer: $22/5$ isn't an integer, and modular arithmetic works over integers only.

2. Find an integer x that satisfies: $5x \equiv 1 \pmod{26}$

Search manually by testing $5(1), 5(2), 5(3), \dots \bmod 26$ until you find a remainder of 1.

Answer: After testing, $x=21$ satisfies the equation since $(5 \times 21 = 105 = 4 \times 26 + 1)$.

3. State $5^{-1} \pmod{26}$.

Answer: $5^{-1} \bmod 26 \Rightarrow 21$

4. Compute $P = (22 \times 5^{-1}) \bmod 26$ to recover the plaintext.

Answer: $P = (22 \times 21) \bmod 26 \Rightarrow 20$

(Note: Time is running out so I won't format the rest of the document)

Exercise 6:

k	m	gcd(k,m)	Inverse exists?	$k^{-1} \bmod m$
5	26	1	Yes	21
7	26	1	Yes	15
2	26	2	No	—
13	26	13	No	—
3	26	1	Yes	9
5	25	5	No	—

$k^{-1} \text{ exists mod } m \Leftrightarrow \gcd(k,m) = 1$

Exercise 7 — Why Some Keys Cannot Decrypt

1. C values for $P=1..13$: 2,4,6,8,10,12,14,16,18,20,22,24,0
2. $E(1) = 2 \times 1 = 2$
 $E(14) = 2 \times 14 = 28 \bmod 26 = 2 \Rightarrow$ equal, confirming the collision.
3. No. $C=2$ could have come from $P=1$ or $P=14$ (or others), so you cannot uniquely determine the original plaintext.
4. No, this function cannot be safely decrypted. It's not invertible.
5. $\gcd(2,26) = 2 \neq 1$. Because k shares a common factor with m , multiple plaintexts map to the same ciphertext which is exactly why decryption fails.

Exercise 8 — Build a Complete Multiplication Cipher

- 1.
2. $k^{-1} \bmod 26 = 21$
3. Encrypt $P = 20$: $C=22$.
Decrypt: $P = (22 \times 21) \bmod 26 \Rightarrow 20$ which matches original.

Exercise 9 — Key Generation

- 1.
2. 12 valid keys (this is $\phi(26) = \phi(2) \times \phi(13) = 1 \times 12 = 12$).
3. Invalid keys: 2, 4, 6, 8, 10, 12, 13, 14, 16, 18, 20, 22, 24.
Invalid because they share a common factor with 26 so $\gcd \neq 1$ and no inverse exists.
4. Inverse pairs listed previously:
(1,1),(3,9),(5,21),(7,15),(9,3),(11,19),(15,7),(17,23),(19,11),(21,5),(23,17),(25,25)

Exercise 10 — Sender and Receiver Scenario

1. $\gcd(7,26)=1$ so yes, valid key.
2. $k^{-1} \bmod 26 = 15$
3. Alice uses $k=7$ to encrypt; Bob uses $k^{-1}=15$ to decrypt.
4. $C = (19 \times 7) \bmod 26 = 3$
5. $P = (3 \times 15) \bmod 26 = 45 \bmod 26 = 19$ which is right, recovering the original.
6. **Conceptual:** Bob's key is directly derived from Alice's since it's the modular inverse of k .

Exercise 11 — What Information Must Be Shared?

1. **Public:** m ($=26$) and the encryption algorithm/formula itself.
2. **Secret:** the key k ($=5$).
3. Yes, if the receiver knows both m and k , they can compute k^{-1} .
4. No, k^{-1} never needs to be sent over the network since it's always derivable from k and m alone.

Exercise 12:

Property	Scheme A (Addition)	Scheme B (Multiplication)
Encryption	$C=(P+7) \bmod 26$	$C=(P \times 7) \bmod 26$
Decryption	$P=(C-7) \bmod 26$	$P=(C \times 15) \bmod 26$
Inverse operation	Subtraction	Multiplication by modular inverse
Every key reversible?	Yes	No
Reversibility condition	Always (additive inverse always exists)	$\gcd(k,26)=1$

Exercise 13 — Modular Exponentiation

1. $3^4 = 81$; $81 \bmod 26 = 3$
2. $\text{pow}(3, 4, 26) \Rightarrow 3$ is correct
3. Here it is

P	1	2	3	4	5	6	7	8	9	10	11	12	13
C	1	16	3	22	1	22	9	14	9	16	3	14	13

P	14	15	16	17	18	19	20	21	22	23	24	25
C	14	3	16	9	14	9	22	1	22	3	16	1

4. No, reversing exponentiation is much harder than subtracting or multiplying by an inverse since there's no simple direct formula.
5. Recovering P from $P^k \equiv C \pmod{m}$ requires solving the discrete logarithm problem, which is computationally hard for large m .

Exercise 14 — Comprehensive Final Challenge

Part 1:

- A. $\text{gcd}(12, 29) = 1$ so $k=12$ is valid
- B. $12^{-1} \bmod 29 = 17$
- C. $C = (17 \times 12) \bmod 29 = 1$
- D. $P = (1 \times 17) \bmod 29 = 17$
- E. $P_{\text{original}} = P_{\text{decrypted}} = 17$ it's verified
- F. Conceptual:
 - i. Bob needs 12^{-1} because the encryption used multiplication, not addition.
 - ii. m isn't secret because it's just a public parameter, knowing m alone gives no way to invert the encryption.
 - iii. If $k=2$: $\text{gcd}(2, 29)=1$ (29 is prime, so every k from 1–28 is automatically valid) since unlike mod 26, there'd be no collision problem here.
 - iv. Bob does not need k^{-1} sent to him since he can generate it himself from k and m .

Part 2 — Complete System Implementation:

```
import math

def mod_inverse(k, m):
    for x in range(1, m):
        if (k * x) % m == 1:
            return x
    return None

def encrypt(P, k, m):
    return (P * k) % m

def decrypt(C, k_inv, m):
    return (C * k_inv) % m

m = int(input("Enter modulus m: "))
k = int(input("Enter key k: "))

if math.gcd(k, m) != 1:
    print("Invalid key: gcd(k, m) must be 1.")
else:
    k_inv = mod_inverse(k, m)
    print(f"Inverse of {k} mod {m} is {k_inv}")

P = int(input("Enter plaintext P: "))
C = encrypt(P, k, m)
print(f"Ciphertext C = {C}")
```

```
P_recovered = decrypt(C, k_inv, m)
print(f"Recovered plaintext = {P_recovered}")

assert P == P_recovered, "Decryption failed!"
print("Decryption verified successfully.")
```

Python Screenshots Not Added Since File Attached